

Reviewer Pack — Minimal Rank of Twisted Tensor Product Structures over Commu...

Entry 076d0940ffe9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 04:03:51 UTC

Minimal Rank of Twisted Tensor Product Structures over Communication Complexity of AND-OR Trees

Entry ID: 076d0940ffe9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 04:03:51 UTC

1. Conjecture

Field A (mathematical branch): Twisted Tensor Product Theory **Field B** (complexity object): Communication Complexity (AND-OR Trees)

Statement:

[‘For every AND-OR tree T with N leaves and depth D , there exists a twisted tensor product structure on the associated vector space V_T such that the minimal rank of V_T is polynomially related to both N and D .’, ‘Equivalently, for all AND-OR trees T , $\text{minRank}(V_T) = \Theta(\text{poly}(N, D))$.’, ‘Furthermore, this relationship holds for any twisted tensor product structure induced by a suitable group representation on the leaves of T .’]

Rationale (proposer’s reasoning):

[‘Twisted tensor products offer a general framework for studying noncommutative algebraic structures and their representations. Their minimal rank could provide insights into the complexity of communication tasks.’, ‘The connection between twisted tensor products and communication complexity has not been extensively explored, suggesting potential for novel insights.’, ‘This conjecture aims to bridge these two fields by establishing a quantifiable relationship that can be tested on small instances.’]

Taxonomy category: Twisted Tensor Product - Communication Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d27d6f82bf8af81c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For an AND-OR tree with N leaves and depth $D \leq 40$, if $\text{minRank}(V_T)$ is within a factor of 2 from $\Theta(\text{poly}(N, D))$ across at least 1000 independent random trees, then the conjecture is supported. Falsification occurs if any tree has a $\text{minRank}(V_T)$ outside this range by more than a factor of 2.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "twisted tensor product theory" AND "communication complexity" AND AND-OR trees - "minimal rank" AND twisted tensor product AND vector space - "polynomial relationship" AND minRank AND AND-OR tree

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_and_or_tree(depth, leaves):
    if depth == 0:
        return leaves.pop()
    else:
        left = generate_and_or_tree(depth - 1, leaves)
        right = generate_and_or_tree(depth - 1, leaves)
        return [left, right]

def matrix_multiply(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    result = [[0 for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                result[i][j] += A[i][k] * B[k][j]
    return result

def gaussian_elimination(A, b):
    n = len(A)
    augmented_matrix = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
```

```

        max_row = j
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
        pivot = augmented_matrix[i][i]
        for j in range(i, n+1):
            augmented_matrix[i][j] /= pivot
        for j in range(n):
            if j != i:
                factor = augmented_matrix[j][i]
                for k in range(i, n+1):
                    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
    return [row[-1] for row in augmented_matrix]

def min_rank(V_T):
    try:
        rank = 0
        A = []
        for v in V_T:
            if any(v):
                A.append(v)
        m = len(A)
        n = len(A[0])
        B = [[A[i][j] for j in range(n)] for i in range(m)]
        rank = gaussian_elimination(B, [0]*n).count(0)
        return rank
    except Exception as e:
        print(f"Error in min_rank: {e}")
        return None

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    D = math.ceil(math.log2(n + 1))
    leaves = [random.choice([0, 1]) for _ in range(n)]
    T = generate_and_or_tree(D, leaves)

    V_T = []
    def build_vector(node):
        if isinstance(node, list):
            left, right = node
            build_vector(left)
            build_vector(right)
        else:
            V_T.append([node])
    build_vector(T)

    rank = min_rank(V_T)
    if rank is None:
        return {
            "metric_name": "minRank",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }
    else:
        return {
            "metric_name": "minRank",
            "metric_value": rank,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": None
        }

expected_rank = n**2

```

```

if rank < expected_rank / 2 or rank > expected_rank * 2:
    return {
        "metric_name": "minRank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"rank {rank} is outside the expected range [{expected_rank/2}, {exp
    }

    return {
        "metric_name": "minRank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(metric_values)/len(metric_values):.2f} std=0.00 support=1.00")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(metric_values)/len(metric_values):.2f} std={math.sqrt(1 - support_fraction)}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='rank outside expected range' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ea6e7157.py", line 123, in <module>
    result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ea6e7157.py", line 77, in run_trial
    T = generate_and_or_tree(D, leaves)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ea6e7157.py", line 23, in generate_and_or_tree
    right = generate_and_or_tree(depth - 1, leaves)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ea6e7157.py", line 22, in generate_and_
    left = generate_and_or_tree(depth - 1, leaves)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ea6e7157.py", line 22, in generate_and_
    left = generate_and_or_tree(depth - 1, leaves)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ea6e7157.py", line 22, in generate_and_
    left = generate_and_or_tree(depth - 1, leaves)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
[Previous line repeated 1 more time]
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ea6e7157.py", line 23, in generate_and_
    right = generate_and_or_tree(depth - 1, leaves)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ea6e7157.py", line 20, in generate_and_
    return leaves.pop()
    ^^^^^^^^^^^^^

```

IndexError: pop from empty list

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11771
2	propose	ollama_remote	glm4:latest	0	0	12540
3	preregistration	ollama_remote	glm4:latest	0	0	6599
4	novelty	ollama_remote	glm4:latest	0	0	4717
5	novelty	ollama_remote	glm4:latest	0	0	12705
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13569
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24735
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11276
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14703
10	judge	ollama_remote	glm4:latest	0	0	49310

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 161925 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/076d0940ffe9.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/076d0940ffe9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/076d0940ffe9.tar.gz` (if generated)